

Grupo 3

Benjamin Elias Saragusti

4° 2

Computación

Turno Mañana

Farmacity

09/09/2025

El Martes empezamos la organización del proyecto creando el kanban Trello, en el cual nos pusimos los objetivos del Código de MYSQL, el DER, y la parte de python. Luego se creó el repositorio de github en el cual nos metimos todos los integrantes y creamos la rama de SQL. Después empezamos a trabajar en el DER contando con las tablas de roles, Clientes, Medicamentos, Empleados, Ventas, medicamentos de góndolas y de la parte posterior. Además hicimos las primeras partes del código en MYSQL contándolo como base para posterior utilización.

```
Drop database if exists Farmacity;
```

```
create database Farmacity;
```

```
USE Farmacity;
```

```
create table Clientes (
```

```
    id_cliente int (3) unsigned auto_increment primary key,
```

```
    Nombre varchar (50),
```

```
    Apellido varchar (50),
```

```
    DNI varchar (20),
```

```
    historial_compras int(3),
```

```
    direccion varchar (200),
```

```
    edad int(3)
```

```
);
```

```
CREATE table Categorías (
```

```
    Id_categoria int(3) unsigned auto_increment primary key,
```

```
    descripcion varchar (100)
```

);

create table Medicamentos (

 Id_medicamento int (3) unsigned auto_increment primary key,

 Nombre varchar (50),

 id_cate int (3) unsigned,

 Precio decimal (10,2),

 stock int(3),

 codigo_barra varchar (25),

 foreign key (id_cate) references Categorias (Id_categoria)

);

create table Elementos_gondola (

 Id_gondola int (3) unsigned auto_increment primary key,

 Id_medicamentos int (3) unsigned,

 Descripcion varchar (100),

 foreign key (Id_medicamentos) references Medicamentos (Id_medicamento)

);

create table Elementos_Fondo (

 Id_Fondo int (3) unsigned auto_increment primary key,

 Id_medicamen int (3) unsigned,

 Descripcion varchar (100),

 foreign key (Id_medicamen) references Medicamentos (Id_medicamento)

);

create table Ventas (

 Id_venta int (3) unsigned auto_increment primary key,

 Id_medi int (3) unsigned,

 Id_cli int (3) unsigned,

 receta varchar (100),

 fecha date,

 cantidad int,

 foreign key (Id_medi) references Medicamentos (Id_medimento),

 foreign key (Id_cli) references Clientes (id_cliente)

);

CREATE table Roles (

 Id_rol int(3) unsigned auto_increment primary key,

 descripcion varchar (100)

);

create table Empleados (

 Id_empleado int (3) unsigned auto_increment primary key,

 nombre varchar (50),

 Id_rols int (3) unsigned,

 foreign key (Id_rols) references Roles (Id_rol)

);

10/09/2025

Se revisó el DER por parte del profesor Federico Bouzon contando con correcciones y algunos cambios a luego aplicar.

12/09/2025

Se corrigió el DER organizándolo por partes, agregando la tabla de características y algunos atributos a las demás tablas. Se empezará la revisión del código para incorporar las correcciones realizadas. Ya se crearon las tablas en el código, contando con todo el contenido excepto los inserts y las consultas.

Se reorganizó la carpeta juntando todos los archivos dentro de una carpeta para mejor acceso. Ya se terminó de redactar las consultas y la organización de la carpeta. Tuve que usar para esto los conocimientos adquiridos por parte de Juan Mclovin, Juan Benicio y Federico Bouzón. Además de la página w3schools.

```
drop database if exists Farmacity;
```

```
create database Farmacity;
```

```
Use Farmacity;
```

```
create table Clientes (
```

```
    Id_cliente int(3) unsigned auto_increment primary key,  
    Nombre varchar (50),  
    Apellido varchar (50),  
    DNI varchar (20),  
    Localidad varchar (200),  
    telefono varchar (20)  
);
```

```
create table Categorias (  
    Id_Categoria int(3) unsigned auto_increment primary key,  
    descripcion varchar (100)  
);
```

```
create table Medicamentos (  
    Id_medicamento int (3) unsigned auto_increment primary key,  
    Id_Cate int(3) unsigned ,  
    Nombre varchar (50),  
    Precio decimal (10,2),  
    stock int(3),  
    codigo_barra varchar (25),  
    receta enum ("SI","NO"),  
    fecha_caducidad date,  
    fecha_produccion date,  
    foreign key (Id_Cate) references Categorias (Id_Categoria)  
);
```

```
create table Elementos_gondola (  
    Id_gondola int (3) unsigned auto_increment primary key,  
    Id_medicamentos int (3) unsigned,  
    Descripcion varchar (100),  
    foreign key (Id_medicamentos) references Medicamentos (Id_medicamento)  
);
```

```
create table Elementos_Fondo (  
    Id_Fondo int (3) unsigned auto_increment primary key,  
    Id_medicamen int (3) unsigned,  
    Descripcion varchar (100),  
    foreign key (Id_medicamen) references Medicamentos (Id_medicamento)  
);
```

```
create table Historial_compras (  
    Id_compras int (3) unsigned auto_increment primary key,  
    Id_client int (3) unsigned,  
    Id_med int (3) unsigned,  
    fecha date,  
    foreign key (Id_med) references Medicamentos (Id_medicamento),  
    foreign key (Id_client) references Clientes (Id_cliente)  
);
```

```
create table Roles (  

```

```
    Id_rol int(3) unsigned auto_increment primary key,  
    descripcion varchar (100)  
);
```

```
create table Empleados (  
    Id_empleado int(3) unsigned auto_increment primary key,  
    Id_role int(3) unsigned,  
    Nombre varchar (50),  
    Sueldo decimal (10,2),  
    Sector varchar (200),  
    foreign key (Id_role) references Roles (Id_rol)  
);
```

```
create table Ventas (  
    Id_venta int(3) unsigned auto_increment primary key,  
    Id_medicamentos int(3) unsigned,  
    Id_clientes int(3) unsigned,  
    Id_empleados int(3) unsigned,  
    Fecha_yhora datetime,  
    cantidad int (4),  
    foreign key (Id_medicamentos) references Medicamentos (Id_medicamento),  
    foreign key (Id_clientes) references Clientes (Id_cliente),  
    foreign key (Id_empleados) references Empleados (Id_empleado)  
);
```


15/09/2025

Corrección del der, está correcto hasta la fecha y el código también con pequeñas modificaciones con respecto de teléfono y dni con int. Ya que está en varchar y se requiere cambiar a int (11)

El resto ya está completo así que por ahora no hay más que hacer.

Ya corregí lo de int (11).

```
Id_cliente int(3) unsigned auto_increment primary key,  
Nombre varchar (50),  
Apellido varchar (50),  
DNI int(11),  
Localidad varchar (200),  
telefono int(11)  
);
```

create table Categorías (

////////////////////////////////////

El código hay que añadirle el relacionamiento con marcas que es la nueva tabla que se debe crear en el der.

Ya establecí aprox el orden en el cual se deben hacer las cosas.

Ayuda

Continúe con creando nuevos subtítulos adheridos al tema del trello

16/09/2025

Agregue marcas al codigo sql.

```
descripcion varchar (100));create table Marcas (      Id_marca int(3) unsigned
auto_increment primary key,  nombre varchar (100));create table Medicamentos (
Id_medicamento int (3) unsigned auto_increment primary key,  Id_Cate int(3)
unsigned ,  id_marc int (3) unsigned,  Nombre varchar (50),  Precio decimal (10,2),
stock int(3),  codigo_barra varchar (25),  receta enum ("SI","NO"),
fecha_caducidad date,  fecha_produccion date,  foreign key (Id_Cate) references
Categorias (Id_Categoria), foreign key (id_marc) references Marcas (Id_marca));create
table Elementos_gondola (
```

17/09/2025

Se corrigio el der, quedando aprobado actualmente y el codigo sql tambien. Asi que por mi parte ya esta hecho lo de sql. Despues cuando cardenas y Ferramola terminen los inserts revisare como quedaron.

Borre el archivo de Cardozo por ser considerado inadecuado.

23/09/2025

Revise y corriji los inserts de cardenas y rosato y acomode el trello

03/10/2025

Se inserto Cassandra canzinos al proyecto en reemplazo de cardoso por lo que elimine los inserts de este ultimo para no usar trabajo de integrantes fuera del equipo, corriji los

inserts de ferramola, normalice el campo sector volviendolo una tabla por separado e indicandole a pedraza que actualize el der. Ademas le indique a canzinos que hiciese los inserts de sectores, roles, elementos gondola y fondo, ademas de corregirlos y adherirlos al codigo principal

05/10/2025

Corregi e inserte los inserts de ferramola de historial de compras en el cual se confundido el tipo date con datetime poniendo ademas de la fecha la hora. Adjunte estos archivos al codigo principal.

06/10/2025

Divide el diccionario para que los chicos lo hagan porque el profe me dijo que ya habia hecho mucho, despues estuve revisando como quedo y ayudando a canzinos y a los demás guiándolos en como se divide todo

14/10/2025

Corregi las consultas y estas ahora son funcionales

```
select Medicamentos.Nombre, sum(Ventas.cantidad) as "Cantidad total"
```

```
from Medicamentos
```

```
Inner join Ventas on Ventas.Id_medicamentos=Medicamentos.Id_medicamento
```

```
group by Id_medicamento
```

```
order by "Cantidad total" DESC
```

```
limit 5;
```

```
Select Nombre as "Medicamento critico", stock as "Stock del medicamento"
```

```
from Medicamentos
```

```
where stock<10;
```

```
select Clientes.nombre as "Nombre de clientes", count(Historial_compras.Id_compras)
```

```
as "Compras totales"
```

```
from Clientes
```

```
inner join Historial_compras on Historial_compras.Id_client=Clientes.Id_cliente
```

```
group by Historial_compras.Id_client
```

```
having "Compras totales">5;
```

21/10/2025

Hice la parte de mysql con python con ayuda de cassandra.

```
import mysql.connector
```

```
import json
```

```
from mysql.connector import errorcode
```

```
cursor = None
```

```
cnx = None
```

```
def ConectarBase():
```

```
    global cnx, cursor
```

```
    try:
```

```
        cnx = mysql.connector.connect(user="root", password="", host="Localhost",
```

```
        database="Farmacity")
```

```

        cursor = cnx.cursor(dictionary=True)

        print('Conexión establecida')

    except mysql.connector.Error as err:

        if err.errno == errorcode.ER_ACCESS_DENIED_ERROR:

            print('Usuario o contraseña incorrectos!')

        elif err.errno == errorcode.ER_BAD_DB_ERROR:

            print('La base de datos no existe!')

        else:

            print(err)

def ConsultaInsertar(meds,clients,employes,datexd, cantidad):

    sql = "INSERT INTO Ventas

(Id_medicamentos,Id_Clientes,Id_Empleados,Fecha_yhora,cantidad)VALUES( %s,

%s, %s, %s,%s)"

    cursor.execute(sql,(meds,clients,employes,datexd,cantidad))

    cnx.commit()

    return cursor.lastrowid

ConectarBase()

def dar_datos():

    medicamentos=int(input("Ingrese el id del mdeicamento que se vendio: "))

    clientes = int(input("Ingrese el id del cliente que compro: "))

    empleados = int(input("Ingrese el id del empleado que realizo la venta: "))

    fechas = (input("Ingrese la fecha y hora en la que se hizo la venta: "))

    cantidades=int(input("Ingrese la cantidad del producto que se compro: "))

    ConsultaInsertar(medicamentos,clientes,empleados,fechas,cantidades)

```

```

def Medicamentos():

    consulta="select * from Medicamentos;"

    cursor.execute(consulta)

    return cursor.fetchall()

def Binario():

    Medsca=Medicamentos()

    longitud=len(Medsca)

    Mitad=longitud//2

    ingrese=(int(input("Ingrese el id del ID el cual quiera verificar: "))-1)

    if ingrese<=Mitad:

        for i in range (Mitad):

            if i==ingrese:

                print (f'Este es el medicamento respectivo {Medsca[ingrese]}')

                break

    else:

        for x in range (Mitad,longitud):

            if x==ingrese:

                print (f'Este es el medicamento respectivo {Medsca[ingrese]}')

                break

Binario()

def proximos_a_vencer():

    consulta="select * from Medicamentos order by fecha_caducidad ASC limit 3;"

    cursor.execute(consulta)

    return cursor.fetchall()

def crear_Archivo_Proximos_Vencer():

```

```

nombre_archivo = "Consulta1.json"

Meds_prox_a_vencer = proximos_a_vencer()

for x in range (len(Meds_prox_a_vencer)):

    Meds_prox_a_vencer[x]["Precio"] = int(Meds_prox_a_vencer[x]["Precio"])

Meds_prox_a_vencer[x]["fecha_caducidad"]=str(Meds_prox_a_vencer[x]["fecha_cadu
cidad"])

    Meds_prox_a_vencer[x]["fecha_produccion"] =
str(Meds_prox_a_vencer[x]["fecha_produccion"])

    with open(nombre_archivo, 'w', encoding='utf-8') as archivo:

        json.dump(Meds_prox_a_vencer, archivo, indent=4, ensure_ascii=False)

crear_Archivo_Proximos_Vencer()

def ventas():

    consulta = "select Empleados.Nombre, count(Ventas.Id_venta) from Empleados
inner join Ventas on Ventas.Id_Empleados=Empleados.Id_empleado group by
Ventas.Id_Empleados;"

    cursor.execute(consulta)

    return cursor.fetchall()

def crear_Archivo_ventas():

    nombre_archivo2 = "Consulta2.json"

    empleadatos=ventas()

    with open(nombre_archivo2, 'w', encoding='utf-8') as archivo2:

        json.dump(empleadatos, archivo2, indent=4, ensure_ascii=False)

crear_Archivo_ventas()

```

31/10/2025

Con cassandra terminamos otra parte del codigo de python

```
import mysql.connector
```

```
import json
```

```
from mysql.connector import errorcode
```

```
import hashlib
```

```
cursor = None
```

```
cnx = None
```

```
def ConectarBase():
```

```
    global cnx, cursor
```

```
    try:
```

```
        cnx = mysql.connector.connect(user="root", password="", host="Localhost",
```

```
        database="Farmacity")
```

```
        cursor = cnx.cursor(dictionary=True)
```

```
        print('Conexión establecida')
```

```
    except mysql.connector.Error as err:
```

```
        if err.errno == errorcode.ER_ACCESS_DENIED_ERROR:
```

```
            print('Usuario o contraseña incorrectos!')
```

```
        elif err.errno == errorcode.ER_BAD_DB_ERROR:
```

```
            print('La base de datos no existe!')
```

```
        else:
```

```
            print(err)
```

```
ConectarBase()
```



```

def sortear_meds():

    consulta="select Nombre, Precio, Stock from medicamentos where Id_Cate= "

    categoria=str(input("Ingrese el id de la categoria que quiera buscar: "))

    consulta=consulta+categoria+" ;"

    cursor.execute(consulta)

    return cursor.fetchall()

def mostrar_sortear():

    A_mostrar=sortear_meds()

    for i in range(len(A_mostrar)):

        print (A_mostrar[i])


def Cod_barras():

    consulta="select Nombre, Precio, Stock,codigo_barra from medicamentos where

codigo_barra= "

    barras=str(input("Ingrese el codigo de barras del producto: "))

    consulta=consulta+barras+" ;"

    cursor.execute(consulta)

    return cursor.fetchall()

def mostrar_barras():

    A_mostrar=Cod_barras()

    for i in range(len(A_mostrar)):

        print (A_mostrar[i])

y añadimos cositas a esto:

amox="personas alérgicas a la amoxicilina, a otras penicilinas o a antibióticos

betalactámicos"

```

sert="no debe utilizarse en personas con alergia a la sustancia o sus componentes, en menores de 6 años"

peni="personas con antecedentes de alergias graves a este medicamento, especialmente anafilaxia, enfermedad del suero u otras reacciones alérgicas previas."

flux="hipersensibilidad al fármaco y en combinación con inhibidores de la monoaminoxidasa (IMAO), y requiere precaución en varias condiciones médicas y situaciones especiales, incluyendo embarazo, lactancia, enfermedades hepáticas y trastornos psiquiátricos activos."

orfi="hipersensibilidad a las benzodiacepinas, insuficiencia respiratoria grave, apnea del sueño, miastenia gravis y en ciertas condiciones del embarazo y lactancia."

sintrom="sangrado activo, úlceras pépticas, insuficiencia hepática o renal grave, hipersensibilidad al acenocumarol, embarazo y ciertas condiciones médicas que aumentan el riesgo de hemorragia."

nolotil="con alergia al metamizol, antecedentes de agranulocitosis, asma, problemas de médula ósea, porfiria hepática aguda, insuficiencia renal o hepática grave, embarazo en el tercer trimestre y lactancia."

def dar_datos():

medicamentos=int(input("Ingrese el id del mdeicamento que se vendio: "))

clientes = int(input("Ingrese el id del cliente que compro: "))

empleados = int(input("Ingrese el id del empleado que realizo la venta: "))

fechas = (input("Ingrese la fecha y hora en la que se hizo la venta: "))

cantidades=int(input("Ingrese la cantidad del producto que se compro: "))

if medicamentos==3:

print (amox)

elif medicamentos==4:

```

        print(sert)

elif medicamentos==6:

    print (peni)

elif medicamentos==7:

    print (flux)

elif medicamentos==11:

    print (orfi)

elif medicamentos==12:

    print (sintrom)

elif medicamentos==15:

    print(nolotil)

```

tambien cambie los inserts de codigo de barras de medicamento a hash

```

codigo_barra varchar (50),

receta enum ("SI","NO"),

fecha_caducidad date,

fecha_produccion date,

@@ -89,21 +89,21 @@ create table Medicamentos (

);

```

insert into Medicamentos

```

(Id_Cate,id_marc,Nombre,Precio,stock,codigo_barra,receta,fecha_caducidad,fecha_pro
duccion)values

(1,1,"Paracetamol",150.50,100,"1740342218120922895","NO","2026-05-20","2024-05
-20"),

```

(3,1,"Ibuprofeno", 220.00, 80, "-8713080813479264554", "NO", "2026-06-15",
"2024-06-15"),
(2,2,"Amoxicilina", 350.00, 50, "-5577755049116257984", "SI", "2026-07-10",
"2024-07-10"),
(4,2,"Sertralina", 500.00, 40, '7610619782785630282', "SI", "2026-08-05",
"2024-08-05"),
(5,3,"Loratadina", 180.75, 70, "-8394422953467208452", 'NO', '2026-09-01',
'2024-09-01'),
(2,3,'Penicilina', 400.00, 60, '6751206841395751602', 'SI', '2026-10-12', '2024-10-12'),
(4,4,'Fluoxetina', 520.00, 30, '-8793214282570593203', 'SI', '2026-11-03', '2024-11-03'),
(3,4,'Diclofenaco', 260.00, 55, '4146393776577963922', 'NO', '2026-12-25',
'2024-12-25'),
(5,5,'Cetirizina', 200.00, 90, '-3537071259406075956', 'NO', '2027-01-15',
'2025-01-15'),
(1,5,'Aspirina', 130.00, 120, '3635816633928414321', 'NO', '2027-02-20', '2025-02-20'),
(4,6,"Orfidal" , 1000 , 97 , '-7855383894576344868', "SI" , '2029-12-20' , '2024-01-10'
),
(3,1,"Sintrom", 2500 , 20 , '-1402941572191463382' , 'SI' , '2030-06-10' , '2018-12-20'),
(8,5,"Orfidil" , 1233 , 50 , '8917533523572325195' , 'NO' , '2020-12-10' , '2016-06-10'),
(3,9,"Ventonil" , 2334 , 123 , '7657069604340486898' , 'NO' , '2019-02-18' ,
'2012-06-12'),
(2,6,"Nolotil" , 4000 , 35 , '-5648408981965590896' , 'SI' , '2019-02-18' , '2014-12-12');
03/11/2025

Termine el menu con un poco de ayuda de cassandra, logrando finalizar toda la parte de
python

```

import mysql.connector

import json

from mysql.connector import errorcode

import time

cursor = None

cnx = None


def ConectarBase():

    global cnx, cursor

    try:

        cnx = mysql.connector.connect(user="root", password="", host="Localhost",
database="Farmacity")

        cursor = cnx.cursor(dictionary=True)

        print('Conexión establecida')


    except mysql.connector.Error as err:

        if err.errno == errorcode.ER_ACCESS_DENIED_ERROR:

            print('Usuario o contraseña incorrectos!')

        elif err.errno == errorcode.ER_BAD_DB_ERROR:

            print('La base de datos no existe!')

        else:

            print(err)


def ConsultaInsertar(meds,clients,employes,datexd, cuantidity):

```

```
sql = "INSERT INTO Ventas  
(Id_medicamentos,Id_Clientes,Id_Empleados,Fecha_yhora,cantidad)VALUES( %s,  
%s, %s, %s,%s)"  
  
cursor.execute(sql,(meds,clients,employes,datexd,cuantity))  
  
cnx.commit()  
  
return cursor.lastrowid
```

ConectarBase()

amox="personas alérgicas a la amoxicilina, a otras penicilinas o a antibióticos
betalactámicos"

sert="no debe utilizarse en personas con alergia a la sustancia o sus componentes, en
menores de 6 años"

peni="personas con antecedentes de alergias graves a este medicamento, especialmente
anafilaxia, enfermedad del suero u otras reacciones alérgicas previas."

flux="hipersensibilidad al fármaco y en combinación con inhibidores de la
monoaminoxidasa (IMAO), y requiere precaución en varias condiciones médicas y
situaciones especiales, incluyendo embarazo, lactancia, enfermedades hepáticas y
trastornos psiquiátricos activos."

orfi="hipersensibilidad a las benzodiacepinas, insuficiencia respiratoria grave, apnea
del sueño, miastenia gravis y en ciertas condiciones del embarazo y lactancia."

sintrom="sangrado activo, úlceras pépticas, insuficiencia hepática o renal grave,
hipersensibilidad al acenocumarol, embarazo y ciertas condiciones médicas que
aumentan el riesgo de hemorragia."

nolotil="con alergia al metamizol, antecedentes de agranulocitosis, asma, problemas de
médula ósea, porfiria hepática aguda, insuficiencia renal o hepática grave, embarazo en
el tercer trimestre y lactancia."

```

def dar_datos():

    medicamentos=int(input("Ingrese el id del mdeicamento que se vendio: "))

    clientes = int(input("Ingrese el id del cliente que compro: "))

    empleados = int(input("Ingrese el id del empleado que realizo la venta: "))

    fechas = (input("Ingrese la fecha y hora en la que se hizo la venta: "))

    cantidades=int(input("Ingrese la cantidad del producto que se compro: "))

    if medicamentos==3:

        print (amox)

    elif medicamentos==4:

        print(sert)

    elif medicamentos==6:

        print (peni)

    elif medicamentos==7:

        print (flux)

    elif medicamentos==11:

        print (orfi)

    elif medicamentos==12:

        print (sintrom)

    elif medicamentos==15:

        print(nolotil)

    ConsultaInsertar(medicamentos,clientes,empleados,fechas,cantidades)

def Medicamentos():

    consulta="select * from Medicamentos;"

    cursor.execute(consulta)

    return cursor.fetchall()

```

```

def Binario():

    Medsca=Medicamentos()

    longitud=len(Medsca)

    Mitad=longitud//2

    ingrese=(int(input("Ingrese el id del ID el cual quiera verificar: "))-1)

    if ingrese<=Mitad:

        for i in range (Mitad):

            if i==ingrese:

                print (f'Este es el medicamento respectivo {Medsca[ingrese]}')

                break

    else:

        for x in range (Mitad,longitud):

            if x==ingrese:

                print (f'Este es el medicamento respectivo {Medsca[ingrese]}')

                break

def proximos_a_vencer():

    consulta="select * from Medicamentos order by fecha_caducidad ASC limit 3;"

    cursor.execute(consulta)

    return cursor.fetchall()

def crear_Archivo_Proximos_Vencer():

    nombre_archivo = "Consulta1.json"

    Meds_prox_a_vencer = proximos_a_vencer()

    for x in range (len(Meds_prox_a_vencer)):

        Meds_prox_a_vencer[x]["Precio"] = int(Meds_prox_a_vencer[x]["Precio"])

```



```
Meds_prox_a_vencer[x]["fecha_caducidad"]=str(Meds_prox_a_vencer[x]["fecha_caducidad"])
```

```
    Meds_prox_a_vencer[x]["fecha_produccion"] =  
str(Meds_prox_a_vencer[x]["fecha_produccion"])
```

```
    with open(nombre_archivo, 'w', encoding='utf-8') as archivo:
```

```
        json.dump(Meds_prox_a_vencer, archivo, indent=4, ensure_ascii=False)
```

```
def ventas():
```

```
    consulta = "select Empleados.Nombre, count(Ventas.Id_venta) from Empleados  
inner join Ventas on Ventas.Id_Empleados=Empleados.Id_empleado group by  
Ventas.Id_Empleados;"
```

```
    cursor.execute(consulta)
```

```
    return cursor.fetchall()
```

```
def crear_Archivo_ventas():
```

```
    nombre_archivo2 = "Consulta2.json"
```

```
    empleadiatos=ventas()
```

```
    with open(nombre_archivo2, 'w', encoding='utf-8') as archivo2:
```

```
        json.dump(empleadiatos, archivo2, indent=4, ensure_ascii=False)
```

```
def sortear_meds():
```

```
    consulta="select Nombre, Precio, Stock from medicamentos where Id_Cate= "
```

```
    categoria=str(input("Ingrese el id de la categoria que quiera buscar: "))
```

```
    consulta=consulta+categoria+" ;"
```

```
    cursor.execute(consulta)
```

```
    return cursor.fetchall()
```

```
def mostrar_sortear():
```

```

A_mostrar=sortear_meds()

for i in range(len(A_mostrar)):

    print (A_mostrar[i])

def Cod_barras():

    consulta="select Nombre, Precio, Stock,codigo_barra from medicamentos where

codigo_barra= "

    barras=str(input("Ingrese el codigo de barras del producto: "))

    consulta=consulta+barras+" ;"

    cursor.execute(consulta)

    return cursor.fetchall()

def mostrar_barras():

    A_mostrar=Cod_barras()

    for i in range(len(A_mostrar)):

        print (A_mostrar[i])

#Dar datos

#Binario

#crear_Archivo_Proximos_Vencer

#crear_Archivo_ventas

#mostrar_sortear

#mostrar_barras

def Menu():

    counter=0

    while counter==0:

        time.sleep(0.25)

        print("////////////////////////////////////")

```

```

time.sleep(0.25)

print("// 1. Para registrar ventas  ////")

time.sleep(0.25)

print("// 2. Busqueda de producto  //")

time.sleep(0.25)

print("/// 3. Imprimir productos a vencer ///")

time.sleep(0.25)

print("// 4. Imprimir ventas por empleados /")

time.sleep(0.25)

print("// 5. Seleccionar Meds por Categoria/")

time.sleep(0.25)

print("// 6. Seleccionar Meds por Barras /")

time.sleep(0.25)

print("// Otro para salir  //")

time.sleep(0.25)

print("////////////////////////////////////")

time.sleep(0.25)

opcion=int (input(""))

if opcion==1:

    dar_datos()

elif opcion==2:

    Binario()

elif opcion==3:

    crear_Archivo_Proximos_Vencer()

    print("Se ha creado el archivo correctamente")

```

```

elif opcion==4:

    crear_Archivo_ventas()

    print("Se ha creado el archivo correctamente")

elif opcion==5:

    mostrar_sortear()

elif opcion==6:

    mostrar_barras()

else:

    print ("Adios que tenga un buen dia")

    break

```

Menu()

08/11/2025

Le saque las mayusculas al inicio de la funciones, elimine los str inputs, le añadi las descripciones a las variables, cierra la base de datos al final del codigo. Elimine redundancias o codigo basura.

```

import mysql.connector
import json
from mysql.connector import errorcode
import time
cursor = None
cnx = None

def conectarBase():
    global cnx, cursor
    try:
        cnx = mysql.connector.connect(user="root", password="",
host="Localhost", database="Farmacity")
        cursor = cnx.cursor(dictionary=True)
        print('Conexión establecida')

    except mysql.connector.Error as err:

```

```

        if err.errno == errorcode.ER_ACCESS_DENIED_ERROR:
            print('Usuario o contraseña incorrectos!')
        elif err.errno == errorcode.ER_BAD_DB_ERROR:
            print('La base de datos no existe!')
        else:
            print(err)
def consultaInsertar(meds,clients,employes,datexd, cantidad):
    # meds: El id del medicamento
    #clients: El id del cliente
    #employes: El id del empleado
    #datexd: La fecha de la compra con la hora
    #cantidad: La cantidad del producto comprado
    #sql: El insert a la base de datos
    sql = "INSERT INTO Ventas
(Id_medicamentos,Id_Clientes,Id_Empleados,Fecha_yhora,cantidad)VALU
ES( %s, %s, %s, %s,%s)"
    cursor.execute(sql, (meds,clients,employes,datexd,cantidad))
    cnx.commit()
    return cursor.lastrowid
conectarBase()
amox="personas alérgicas a la amoxicilina, a otras penicilinas o a
antibióticos betalactámicos"
sert="no debe utilizarse en personas con alergia a la sustancia o
sus componentes, en menores de 6 años"
peni="personas con antecedentes de alergias graves a este
medicamento, especialmente anafilaxia, enfermedad del suero u otras
reacciones alérgicas previas."
flux="hipersensibilidad al fármaco y en combinación con inhibidores
de la monoaminoxidasa (IMAO), y requiere precaución en varias
condiciones médicas y situaciones especiales, incluyendo embarazo,
lactancia, enfermedades hepáticas y trastornos psiquiátricos
activos."
orfi="hipersensibilidad a las benzodiacepinas, insuficiencia
respiratoria grave, apnea del sueño, miastenia gravis y en ciertas
condiciones del embarazo y lactancia."
sintrom="sangrado activo, úlceras pépticas, insuficiencia hepática
o renal grave, hipersensibilidad al acenocumarol, embarazo y
ciertas condiciones médicas que aumentan el riesgo de hemorragia."
nolotil="con alergia al metamizol, antecedentes de agranulocitosis,
asma, problemas de médula ósea, porfiria hepática aguda,
insuficiencia renal o hepática grave, embarazo en el tercer
trimestre y lactancia."
def dar_datos():

```

```

#medicamentos: El id del medicamento
#clientes: El id del cliente
#empleados: El id del empleado
#fechas: La fecha y la hora de la venta
#cantidades: La cantidad del producto vendido
medicamentos=int(input("Ingrese el id del medicamento que se
vendio: "))
clientes = int(input("Ingrese el id del cliente que compro: "))
empleados = int(input("Ingrese el id del empleado que realizo
la venta: "))
fechas = input("Ingrese la fecha y hora en la que se hizo la
venta: ")
cantidades=int(input("Ingrese la cantidad del producto que se
compro: "))
if medicamentos==3:
    print (amox)
elif medicamentos==4:
    print(sert)
elif medicamentos==6:
    print (peni)
elif medicamentos==7:
    print (flux)
elif medicamentos==11:
    print (orfi)
elif medicamentos==12:
    print (sintrom)
elif medicamentos==15:
    print(nolotil)

consultaInsertar(medicamentos,clientes,empleados,fechas,cantidades)
def medicamentos():
    #consulta: La consulta a la base de datos para tener los
registros de la tabla medicamentos
    consulta="select * from Medicamentos;"
    cursor.execute(consulta)
    return cursor.fetchall()
def binario():
    #Medsca: tiene los resultados de medicamentos, contando los
registros
    #longitud: contiene el largo del diccionario de los
medicamentos
    # Mitad: Es la mitad entera de la cantidad de registros

```

```

    # ingreso: El Id del medicamento a ver, se le resta 1 ya que al
pasar de sql a python
    #el primer registro de sql tiene valor de 1 mientras que en
python es 0
    Medsca=medicamentos()
    longitud=len(Medsca)
    Mitad=longitud//2
    ingreso=(int(input("Ingrese el id del medicamento el cual
quiera verificar: "))-1
    if ingreso<Mitad:
        for i in range (Mitad):
            if i==ingreso:
                print (f"Este es el medicamento
respectivo{Medsca[ingreso]}")
                break
    else:
        for x in range (Mitad,longitud):
            if x==ingreso:
                print (f"Este es el medicamento
respectivo{Medsca[ingreso]}")
                break
def proximos_a_vencer():
    #consulta: Selecciona los 3 medicamentos mas cercanos a vencer
de la tabla sql
    consulta="select * from Medicamentos order by fecha_caducidad
ASC limit 3;"
    cursor.execute(consulta)
    return cursor.fetchall()
def crear_Archivo_Proximos_Vencer():
    #nombre_archivo: El nombre del archivo a crear en formato json
    #Meds_prox_a_vencer: Es el diccionario generado por la funcion
proximos a vencer, el cual despues vuelve
    #todos sus valores a str para poderlos pasar al formato
    nombre_archivo = "Consulta1.json"
    Meds_prox_a_vencer = proximos_a_vencer()
    Meds_prox_a_vencer=str(Meds_prox_a_vencer)
    with open(nombre_archivo, 'w', encoding='utf-8') as archivo:
        json.dump(Meds_prox_a_vencer, archivo, indent=4,
ensure_ascii=False)
def ventas():
    #consulta: selecciona los vendedores con su total de ventas
    consulta = "select Empleados.Nombre, count(Ventas.Id_venta)
from Empleados inner join Ventas on

```

```

Ventas.Id_Empleados=Empleados.Id_empleado group by
Ventas.Id_Empleados;"
    cursor.execute(consulta)
    return cursor.fetchall()
def crear_Archivo_ventas():
    #nombre_archivo2: El nombre del archivo a crear en formato json
    #empleadiatos: Es el diccionario generado por la funcion ventas
    nombre_archivo2 = "Consulta2.json"
    empleadiatos=ventas()
    with open(nombre_archivo2, 'w', encoding='utf-8') as archivo2:
        json.dump(empleadiatos, archivo2, indent=4,
ensure_ascii=False)
def sortear_meds():
    #consulta: es la que selecciona los datos de la tabla
medicamentos por id de categoria
    #se completa despues de recibir el id de la categoria. Por eso
se ve sin los datos y ;
    #Categoria: El id de la categoria seleccionada, el cual despues
se agrega a la variable consulta
    #junto con ; para ejecutarla
    consulta="select Nombre, Precio, Stock from medicamentos where
Id_Cate= "
    categoria=input("Ingrese el id de la categoria que quiera
buscar: ")
    consulta=consulta+categoria+" ;"
    cursor.execute(consulta)
    return cursor.fetchall()
def mostrar_sortear():
    #A_mostrar: Contiene el diccionario generado por sortear_meds,
que despues de muestra linea
    #por linea separado.
    A_mostrar=sortear_meds()
    for i in range(len(A_mostrar)):
        print (A_mostrar[i])
def cod_barras():
    #consulta: Es la funcion que muestra los medicamentos segun su
codigo de barras, le falta ese
    #dato ya que el cajero lo inserta y despues se añade.
    #barras: Es el codigo de barras que inserta el cajero, despues
de escribirse se le añade a la variable
    #consulta junto con ; para ejecutar
    consulta="select Nombre, Precio, Stock,codigo_barra from
medicamentos where codigo_barra= "

```


[illegible]

```

opcion=int (input(""))
if opcion==1:
    dar_datos()
elif opcion==2:
    binario()
elif opcion==3:
    crear_Archivo_Proximos_Vencer()
    print("Se ha creado el archivo correctamente")
elif opcion==4:
    crear_Archivo_ventas()
    print("Se ha creado el archivo correctamente")
elif opcion==5:
    mostrar_sortear()
elif opcion==6:
    mostrar_barras()
else:
    if cnx.is_connected():
        cnx.close()
    print ("Adios que tenga un buen dia")
    counter=False

menu ()

```

10/11/2025

Correji las 20 mil variables a una lista de diccionarios:

def aplicar_contra_indicaciones():

#lista: donde se van a almacenar los diccionarios

#contraindicaciones=donde se guardan las contraindicaciones

#ids:donde se guardan los ids

#medicamento: diccionario base para crear los demas

lista=[]

contraindicaciones=["personas alérgicas a la amoxicilina, a otras penicilinas o a antibióticos betalactámicos","no debe utilizarse en personas con alergia a la sustancia o sus componentes, en menores de 6 años",

"personas con antecedentes de alergias graves a este medicamento, especialmente anafilaxia, enfermedad del suero u otras reacciones alérgicas previas.", "hipersensibilidad al fármaco y en combinación con inhibidores de la monoaminooxidasa (IMAO), y requiere precaución en varias condiciones médicas y situaciones especiales, incluyendo embarazo, lactancia, enfermedades hepáticas y trastornos psiquiátricos activos.",

"hipersensibilidad a las benzodiacepinas, insuficiencia respiratoria grave, apnea del sueño, miastenia gravis y en ciertas condiciones del embarazo y lactancia.",

"sangrado activo, úlceras pépticas, insuficiencia hepática o renal grave, hipersensibilidad al acenocumarol, embarazo y ciertas condiciones médicas que aumentan el riesgo de hemorragia.",

"con alergia al metamizol, antecedentes de agranulocitosis, asma, problemas de médula ósea, porfiria hepática aguda, insuficiencia renal o hepática grave, embarazo en el tercer trimestre y lactancia."

]

```
ids=[3,4,6,7,11,12,15]
```

```
for i in range(len(ids)):
```

```
    medicamento = {
```

```
        "id": ids[i],
```

```
        "contraindicacion": contraindicaciones[i]
```

```
    }
```

```

        lista.append(medicamento)

    return lista

y en la parte de dar datos agregue esta correccion

contraindicaciones=aplicar_contra_indicaciones()

for i in range (len(contraindicaciones)):

    if contraindicaciones[i]["id"]==medicament:

        print (contraindicaciones[i]["contraindication"])

```

11/11/2025

Aplique las ultimas correcciones al codigo:

```

import mysql.connector
import json
from mysql.connector import errorcode
import time
import datetime
cursor = None
cnx = None
#conectarbase: Conecta con la base de datos
def conectarBase():
    global cnx, cursor
    try:
        cnx = mysql.connector.connect(user="root", password="", host="Localhost",
database="Farmacity")
        cursor = cnx.cursor(dictionary=True)
        print('Conexión establecida')

    except mysql.connector.Error as err:
        if err.errno == errorcode.ER_ACCESS_DENIED_ERROR:
            print('Usuario o contraseña incorrectos!')
        elif err.errno == errorcode.ER_BAD_DB_ERROR:
            print('La base de datos no existe!')
        else:
            print(err)

#consultainsertar: Se usa de base para hacer los inserts a la tabla ventas
def consultaInsertar(meds,clients,employes,datexd, cantidad):
    # meds: El id del medicamento
    #clients: El id del cliente

```

```

#employes: El id del empleado
#datexd: La fecha de la compra con la hora
#cuantity: La cantidad del producto comprado
#sql: El insert a la base de datos
sql = "INSERT INTO Ventas
(Id_medicamentos,Id_Clientes,Id_Empleados,Fecha_yhora,cantidad)VALUES( %s, %s,
%s, %s,%s)"
cursor.execute(sql,(meds,clients,employes,datexd,cuantity))
cnx.commit()
return cursor.lastrowid

```

#aplicar_contra_indicaciones: tiene la tabla con las contraindicaciones base.

```
def aplicar_contra_indicaciones():
```

```
    #lista: donde se van a almacenar los diccionarios
```

```
    #contraindicaciones=donde se guardan las contraindicaciones
```

```
    #ids:donde se guardan los ids
```

```
    #medicamento: diccionario base para crear los demas
```

```
    lista=[]
```

```
    contraindicaciones=["personas alérgicas a la amoxicilina, a otras penicilinas o a
antibióticos betalactámicos","no debe utilizarse en personas con alergia a la sustancia o
sus componentes, en menores de 6 años",
```

```
        "personas con antecedentes de alergias graves a este medicamento,
especialmente anafilaxia, enfermedad del suero u otras reacciones alérgicas
previas.", "hipersensibilidad al fármaco y en combinación con inhibidores de la
monoaminooxidasa (IMAO), y requiere precaución en varias condiciones médicas y
situaciones especiales, incluyendo embarazo, lactancia, enfermedades hepáticas y
trastornos psiquiátricos activos.",
```

```
        "hipersensibilidad a las benzodiacepinas, insuficiencia respiratoria grave,
apnea del sueño, miastenia gravis y en ciertas condiciones del embarazo y lactancia.",
```

```
        "sangrado activo, úlceras pépticas, insuficiencia hepática o renal grave,
hipersensibilidad al acenocumarol, embarazo y ciertas condiciones médicas que
aumentan el riesgo de hemorragia.",
```

```
        "con alergia al metamizol, antecedentes de agranulocitosis, asma,
problemas de médula ósea, porfiria hepática aguda, insuficiencia renal o hepática grave,
embarazo en el tercer trimestre y lactancia."

```

```
    ]
```

```
    ids=[3,4,6,7,11,12,15]
```

```
    for i in range(len(ids)):
```

```
        medicamento = {
```

```
            "id": ids[i],
```

```
            "contraindicacion": contraindicaciones[i]
```

```
        }
```

```
        lista.append(medicamento)
```

```
    return lista
```

#add_contraindicaciones: Permite añadir contraindicaciones

```
def add_contraindicaciones(Lista_original):  
    #ids: El id del medicamento a volver diccionario  
    #contraindicaciones: la contraindicacion del medicamento  
    #Lista_original: El diccionario original  
    ids=int(input("Ingrese el id del medicamento: "))  
    contraindicaciones=input("Ingrese la contraindicacion")  
    medicamento = {  
        "id": ids,  
        "contraindicacion": contraindicaciones  
    }  
    Lista_original.append(medicamento)  
    return Lista_original
```

#dar_datos: Aqui el cajero da los datos para usando de base consulta insertar se hace un registro para la base de datos

```
def dar_datos(Diccionario):  
    #medicament: El id del medicamento  
    #clientes: El id del cliente  
    #empleados: El id del empleado  
    #fechas: La fecha y la hora de la venta  
    #cantidades: La cantidad del producto vendido  
    #Diccionario: Contiene la lista de diccionarios con las contraindicaciones  
    try:  
        medicament=int(input("Ingrese el id del medicamento que se vendio: "))  
        clientes = int(input("Ingrese el id del cliente que compro: "))  
        empleados = int(input("Ingrese el id del empleado que realizo la venta: "))  
        fechas = datetime.datetime.now()  
        cantidades=int(input("Ingrese la cantidad del producto que se compro: "))  
        for i in range (len(Diccionario)):  
            if Diccionario[i]["id"]==medicament:  
                print (f"El medicamento seleccionado cuenta con la siguiente  
                contraindicacion: {Diccionario[i]['contraindicacion']}")  
                consultaInsertar(medicament,clientes,empleados,fechas,cantidades)  
    except Exception:  
        print ("Se ha equivocado en introducir los datos. Porfavor reintente la inserción")
```

#medicamentos: Realiza una consulta para tener todo el diccionario de medicamentos
def medicamentos():

```
    #consulta: La consulta a la base de datos para tener los registros de la tabla  
medicamentos  
    consulta="select * from Medicamentos;"  
    cursor.execute(consulta)
```

```
return cursor.fetchall()
```

#binario: Realiza una busqueda binaria usando de base el diccionario de la funcion medicamentos

def binario():

#Medsca: tiene los resultados de medicamentos, contando los registros

#longitud: contiene el largo del diccionario de los medicamentos

Mitad: Es la mitad entera de la cantidad de registros

ingrese: El Id del medicamento a ver, se le resta 1 ya que al pasar de sql a python

#el primer registro de sql tiene valor de 1 mientras que en python es 0

Medsca=medicamentos()

longitud=len(Medsca)

Mitad=longitud//2

ingrese=(int(input("Ingrese el id del medicamento el cual quiera verificar: ")))

if ingrese<Mitad:

for i in range (Mitad):

if i==ingrese:

print (f"Este es el medicamento respectivo{Medsca[ingrese]}")

break

else:

for x in range (Mitad,longitud):

if x==ingrese:

print (f"Este es el medicamento respectivo{Medsca[ingrese]}")

break

#proximos_a_vencer: Realiza una consulta conteniendo los datos de los medicamentos

proximos a vencer

def proximos_a_vencer():

#consulta: Selecciona los 3 medicamentos mas cercanos a vencer de la tabla sql

consulta="select * from Medicamentos order by fecha_caducidad ASC limit 3;"

cursor.execute(consulta)

return cursor.fetchall()

#crear_Archivo_Proximos_Vencer: Crea un archivo json con los datos de proximo a vencer

def crear_Archivo_Proximos_Vencer():

#nombre_archivo: El nombre del archivo a crear en formato json

#Meds_prox_a_vencer: Es el diccionario generado por la funcion proximos a vencer, el cual despues vuelve

#todos sus valores a str para poderlos pasar al formato

nombre_archivo = "Consulta1.json"

Meds_prox_a_vencer = proximos_a_vencer()

Meds_prox_a_vencer=str(Meds_prox_a_vencer)

with open(nombre_archivo, 'w', encoding='utf-8') as archivo:

```

    json.dump(Meds_prox_a_vencer, archivo, indent=4, ensure_ascii=False)

#ventas: obtiene los datos de cuantas ventas se realizo por empleado
def ventas():
    #consulta: selecciona los vendedores con su total de ventas
    consulta = "select Empleados.Nombre, count(Ventas.Id_venta) from Empleados inner
join Ventas on Ventas.Id_Empleados=Empleados.Id_empleado group by
Ventas.Id_Empleados;"
    cursor.execute(consulta)
    return cursor.fetchall()

#crear_Archivo_ventas: Crea un archivo json con los datos de ventas
def crear_Archivo_ventas():
    #nombre_archivo2: El nombre del archivo a crear en formato json
    #empleadiatos: Es el diccionario generado por la funcion ventas
    nombre_archivo2 = "Consulta2.json"
    empleadiatos=ventas()
    with open(nombre_archivo2, 'w', encoding='utf-8') as archivo2:
        json.dump(empleadiatos, archivo2, indent=4, ensure_ascii=False)

#sortear_meds: Se buscan medicamentos segun su categoria
def sortear_meds():
    #consulta: es la que selecciona los datos de la tabla medicamentos por id de categoria
    #se completa despues de recibir el id de la categoria. Por eso se ve sin los datos y ;
    #Categoria: El id de la categoria seleccionada, el cual despues se agrega a la variable
    consulta
    #junto con ; para ejecutarla
    consulta="select Nombre, Precio, Stock from medicamentos where Id_Cate= "
    categoria=input("Ingrese el id de la categoria que quiera buscar: ")
    consulta=consulta+categoria+" ;"
    cursor.execute(consulta)
    return cursor.fetchall()

#mostrar_sortear: Se muestran los resultados de sortear_meds
def mostrar_sortear():
    #A_mostrar: Contiene el diccionario generado por sortear_meds, que despues de
    muestra linea
    #por linea separado.
    A_mostrar=sortear_meds()
    for i in range(len(A_mostrar)):
        print (A_mostrar[i])

#cod_barras: Se buscan medicamentos segun su codigo de barras
def cod_barras():

```


#consulta: Es la funcion que muestra los medicamentos segun su codigo de barras, le falta ese

#dato ya que el cajero lo inserta y despues se añade.

#barras: Es el codigo de barras que inserta el cajero, despues de escribirse se le añade a la variable

#consulta junto con ; para ejecutar

```
consulta="select Nombre, Precio, Stock,codigo_barra from medicamentos where  
codigo_barra= "
```

```
barras=input("Ingrese el codigo de barras del producto: ")
```

```
consulta=consulta+barras+" ;"
```

```
cursor.execute(consulta)
```

```
return cursor.fetchall()
```

#mostrar_barras: Se muestran los datos de cod_barras

```
def mostrar_barras():
```

#A_mostrar: Muestra el diccionario del medicamento obtenido despues de haberlo buscado por

#codigo de barra en la funcion cod_barras

```
A_mostrar=cod_barras()
```

```
print (A_mostrar)
```

#menu: Es el menu que contiene todas las funciones llamadas

```
def menu():
```

#counter: Es lo que permite que se repita indefinidamente, cambia de valor al terminarse de ejecutar

#el programa

#opcion: La seleccion del cajero para ver que es lo que requiere hacer

#contraindicaciones: Cuenta con el diccionario de contraindicaciones

```
conectarBase()
```

```
counter=True
```

```
contraindicaciones = aplicar_contra_indicaciones()
```

```
while counter==True:
```

```
    time.sleep(0.25)
```

```
    print("////////////////////////////////////")
```

```
    time.sleep(0.25)
```

```
    print("//  1. Para registrar ventas  //")
```

```
    time.sleep(0.25)
```

```
    print("//  2. Busqueda de producto  //")
```

```
    time.sleep(0.25)
```

```
    print("///  3. Imprimir productos a vencer //")
```

```
    time.sleep(0.25)
```

```
    print("//  4. Imprimir ventas por empleados /")
```

```
    time.sleep(0.25)
```

```
    print("//  5. Seleccionar Meds por Categoria/")
```

```

time.sleep(0.25)
print("// 6. Seleccionar Meds por Barras //")
time.sleep(0.25)
print("// 7. Para añadir contraindicaciones/")
time.sleep(0.25)
print("// Otro para salir //")
time.sleep(0.25)
print("////////////////////////////////////")
time.sleep(0.25)
opcion=int (input(""))
if opcion==1:
    dar_datos(contraindicaciones)
elif opcion==2:
    binario()
elif opcion==3:
    crear_Archivo_Proximos_Vencer()
    print("Se ha creado el archivo correctamente")
elif opcion==4:
    crear_Archivo_ventas()
    print("Se ha creado el archivo correctamente")
elif opcion==5:
    mostrar_sortear()
elif opcion==6:
    mostrar_barras()
elif opcion==7:
    contraindicaciones=add_contraindicaciones(contraindicaciones)
else:
    if cnx.is_connected():
        cnx.close()
    print ("Adios que tenga un buen dia")
    counter=False

```

menu()

Todo el codigo esta bien falta el manual de Pedraza

12/11/2025

Agregue la funcion de cargar las contraindicaciones al JSON asi cuando agruegas una se guarda y no se pierde al cerrar el programa. Si este no detecta el JSON usa la copia de seguridad antigua.

#cargar_datos_json: Guarda las contraindicaciones en un archivo json

def cargar_datos_json():

#datos_cargados: Los datos del archivo json si se carga

#contraindications: En el caso de que el archivo no este se restaura la copia de seguridad.

try:

```
with open("Contraindicaciones.json", 'r', encoding='utf-8') as archivo:
```

```
    datos_cargados = json.load(archivo)
```

```
    print ("Se ha cargado las contraindicaciones del archivo local")
```

```
    return datos_cargados
```

except Exception:

```
    contraindicaciones=aplicar_contra_indicaciones()
```

```
    print ("La carga ha fallado, se restaurara la copia predeterminada")
```

```
    return contraindicaciones
```

en el menu:

```
contraindicaciones = cargar_datos_json()
```

despues del add:

```
with open("Contraindicaciones.json", 'w', encoding='utf-8') as  
archivo:
```

```
    json.dump(contraindicaciones, archivo, indent=4,
```

```
ensure_ascii=False)
```

y por si acaso antes de cerrar:

```
with open("Contraindicaciones.json", 'w', encoding='utf-8') as  
archivo:
```

```
    json.dump(contraindicaciones, archivo, indent=4,
```

```
ensure_ascii=False)
```